

[numpy](#) / Module 1

MODULE 1: NUMPY

Introduction to Numpy

Welcome to this session on introduction to Numpy. Numpy is a library in python used for handling multidimensional arrays. It is a short form for numerical python. Numpy array in a way are like the lists you were introduced to in the introduction to python programming. As the arrays grow bigger numpy array provide efficient ways and operations for managing arrays. Arrays organize data into rows and columns

Numpy provides support for large multidimensional arrays and matrices, with mathematical functions used for operating on this arrays.

To be able to use Numpy in your machine you will need to install it. If you installed anacoda in your environment, it comes with Numpy installed. So you will not need to install it. If you did not you can use

```
pip install numpy
```

Once you have Numpy installed in your machine, we can use it in our notebooks by importing it using the following code

```
import numpy as np
```

As np provides just an alias so that we do not have to type numpy all the times. Lists as discussed earlier can contain elements of different datatypes in contrast numpy arrays contains only elements of one data type. If there are other datatypes in the array, numpy will try casting them.

There are various ways we can create numpy array;

1. Creating numpy array from a list

We can convert a numpy array to a list as follows; lets say we have a list of student marks we can convert it to numpy using the np.array()

```
1 import numpy as np
2 lis_marks=[40,67,89,45,67]
3 numpy_list=np.array(lis_marks)
4 print(numpy_list)
```

```
1 import numpy as np
2 lis_marks=[40,67,89,45,67]
3 numpy_list.dtype
```

You can specify the dtype of the created array by specifying the dtype attribute in the the array function

```
1 import numpy as np
2 lis_marks=[40,67,89,45,67]
3 numpy_list_f=np.array(lis_marks, dtype=float)
4 print(numpy_list_f.dtype)
```

You can see from the result the numpy array has a float data type now. The default one provided from the previous code is an integer.

Creating arrays from scratch We can also create numpy arrays from scratch using the built in functions such as zeros, ones

The zeros() function

The numpy's zeros function return an array filled with zeros. The parameters expected by the function include;

Shape: This specifies the dimension of the array. It is a tuple of two values specifying the number of rows and columns of the array.

dtype: The datatype expected of the array such as int8. The default datatype if not specified is float64

```
1 #creates array of five elements filled with zeros
2 import numpy as np
3 np.zeros(5)
```

```
1 #Creates a 3, by 3 matrix of type int
2 import numpy as np
3 np.zeros((3,3), dtype="int16")
```

```
1 #Creates a 7 by 10 matrix with dtype float
2 import numpy as np
3 np.zeros((7,10), dtype="float64")
```

zeros_like() function, creates an array filled with zeros that has the same attributes as the provided array

```
1 import numpy as np
2 arra_a=np.array([[1,2,3],[5,6,7],[8,9,10]])
3 print(arra_a)
```

```
1 #using the zeros_like to create the same array
2 import numpy as np
3 arra_a=np.array([[1,2,3],[5,6,7],[8,9,10]])
4 array_like=np.zeros_like(arra_a)
5 print(array_like)
```

The ones() function

Similar to the zeros function, the numpy ones() function creates an array of specified dimensions filled with the value 1.

```
1 #creating 4 by 4 matrix filled with ones
2 import numpy as np
3 np.ones((4,4), dtype="int64")
```

ones_like(): This function works in a similar manner as zeros_like as it creates an array filled with ones based on an existing array.

```
1 import numpy as np
2 arra_b=np.array([[23,55,66,77],[67,89,90,56],[50,60,70,80]])
3 print(arra_b)
```

```
1 import numpy as np
2 arra_b=np.array([[23,55,66,77],[67,89,90,56],[50,60,70,80]])
3 arra_bones_like=np.ones_like(arra_b)
4 print(arra_bones_like)
```

full() function:

The full functions fills the specified array with a particular value. For example if you want to create an array that will be filled with the value 3.14, we will use the numpy full function.

parameters The full() function expects the following parameters

Shape: This is the attributes that specifies the structure of the array(The number of rows and columns in the array). It is a tuple of two values. If a single value is provided, it will create a list with the size of the specified length.

fill_value: The value the array will be populated with.

data_type: This indicates the desired datatype for the created array. This is usually optional

A related function `full_like()` creates an array based on the existing array and fills it with the provided values

```
1 import numpy as np
2 array_pi=np.full((3,4), fill_value=3.14)
3 print(array_pi)
```



```
1 import numpy as np
2 array_pi=np.full((3,4), fill_value=3.14)
3 array_pilike=np.full_like(array_pi, fill_value=4.)
4 print(array_pilike)
```

```
1 #converting a list to an array
2 import numpy as np
3 alist_array=np.asarray([23,45,67,42,78], dtype=
4 print(alist_array)
```

asarray() fuction

The asarray() converts the provided input to an array

Parameters of the assarray function

a: This is the input that needs to be converted into ana array

dtype: The datatype to be assigned to the converted array. If it is not specified it is inferred from the input provided

arange() function

This function is similar to the python range() function you learnt in the introduction to programming II the only difference is that it will return an array.

The first number in the function represent the start, the second number represents the stop and the last number represents the step

```
1 import numpy as np
2 array_num=np.arange(30,100,5)
3 print(array_num)
```

We can use the reshape() function to be able to convert the array generated by the arange() function to the dimensions that we want as follows.

```
1 import numpy as np
2 array_num=np.arange(30,100,5)
3 array_num.reshape(2,7)
```

Creating an indetity matrix

To create an indetity matrix, we use the `eye()` function. The `eye` function expects as parameters, the number of rows in the array, the number of columns which if not specified the number of rows is created. The index of the diagonal, if this is not provided 0 will be used which is the main diagonal. The data type of the returned array

```
1 #Creates a 2 by 2 indetity matrix
2 import numpy as np
3 np.eye(2)
```

```
1 #makes the diagonal start at index 1 and creates
2 import numpy as np
3 np.eye(3,4,1)
```

Attributes of ndarray

ndim: Specifies the number of axis in the array

For example, we can check the number of axis that the following array has using the ndim attribute. in our example below, our array has 2 dimensions

```
1 import numpy as np
2 new_array=np.ones((4,5), dtype="int64")
3 print(new_array.ndim)
```

```
1 import numpy as np
2 new_array=np.ones((4,5), dtype="int64")
3 print(new_array)
```

shape: The shape attribute specifies the size of the array in each dimension. Using the array we created in the previous cell, we can see the shape is 4 rows and 5 columns. For each of the column there are 4 elements and for each of the row there are 5 elements.

```
1 import numpy as np
2 new_array=np.ones((4,5), dtype="int64")
3 print(new_array.shape)
```

size: Indicates the total number of elements in the array

```
1 #Total number of elements in the array are 20
2 import numpy as np
3 new_array=np.ones((4,5), dtype="int64")
4 print(new_array.size)
```

dtype: Indicates the types of elements in the array.

```
1 #New array is of type int
2 import numpy as np
3 new_array=np.ones((4,5), dtype="int64")
4 print(new_array.dtype)
```

Basic operations on numpy arrays

The arithmetic operations in the numpy array are done elementwise.

Adding two array

```
1 import numpy as np
2 #adding two arrays
3 array_a=np.array([45,67,89,42])
4 print("First array")
5 print(array_a)
6 array_b=np.arange(4)
7 print("Second array")
8 print(array_b)
9 result=array_a+array_b
10 print("Result of adding first array and second array")
11 print(result)
12 #adding two dimensional array
13 array_c=np.arange(0,20,1).reshape((4,5))
14 print("Third array")
15 print(array_c)
16 array_d=np.arange(20,40,1).reshape((4,5))
17 print("Fourth array")
18 print(array_d)
19 print("Two dimensional result: Adding the two arrays")
20 print(array_c+array_d)
21 print("First array")
22 print(array_a)
23 array_b=np.arange(4)
24 print("Second array")
25 print(array_b)
26 result=array_a+array_b
27 print("Result of adding first array and second array")
28 print(result)
```

Subtracting two array

```
1 import numpy as np
2 array_d=np.arange(20,40,1).reshape((4,5))
3 array_c=np.arange(0,20,1).reshape((4,5))
4 print(array_d-array_c)
```

Multiplying two array

The product `*` in numpy does element wise multiplication. If you want to do matrix multiplication you can use the `@` operator or the `dot` function.


```
1 import numpy as np
2 array_a=np.array([45,67,89,42])
3 array_b=np.arange(4)
4 print(array_a*array_b)
```

```
1 import numpy as np
2 #matrix multiplication
3 array_a=np.array([45,67,89,42])
4 array_b=np.arange(4)
5 print(array_a@array_b)
```

Unary operations

There are functions to implement most of the unary operations in numpy such as computing the sum, finding the minimum element, finding the maximum element.

```
1 import numpy as np
2 array_d=np.arange(20,40,1).reshape((4,5))
3 print(array_d.sum())
```

```
1 import numpy as np
2 array_d=np.arange(20,40,1).reshape((4,5))
3 print(array_d.min())
```

Universal functions

Numpy provides universal functions for doing mathematical calculations such as cos, sin, exp. These functions operate element wise.

```
1 import numpy as np
2 array_a=np.arange(2,20,2)
3 print(np.exp(array_a))
4 print(np.sin(array_a))
```